

Efficient Algorithms for Constructing Very Sparse Spanners and Emulators

Paper by: Michael Elkin and Ofer Neiman

Presented by: Mohammadreza Ahmadi

Supervisor: Dr. Hossein Jowhari

Faculty of Mathematics
K. N. Toosi University of Technology

August 11, 2026

- 1 Introduction and Preliminaries
- 2 Motivation and Theoretical Bounds
- 3 Core Algorithm: Unweighted Graphs
- 4 Extension to Weighted Graphs
- 5 Implications and Skeletons

Core Definitions: Spanners

We consider unweighted, undirected n -vertex graphs $G = (V, E)$.

- **Multiplicative α -Spanner:** A subgraph $H = (V, E')$ is an α -spanner of G if for every pair $u, v \in V$:

$$d_H(u, v) \leq \alpha \cdot d_G(u, v)$$

*(The parameter $\alpha \geq 1$ is called the **stretch** factor).*

- **General (α, β) -Spanner:** A subgraph $H = (V, E')$ is an (α, β) -spanner of G if for every pair $u, v \in V$:

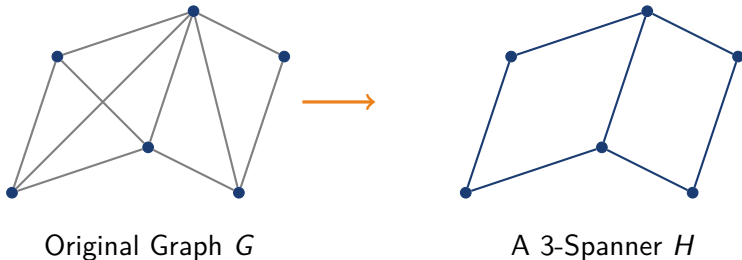
$$d_H(u, v) \leq \alpha \cdot d_G(u, v) + \beta$$

*(If $\alpha = 1 + \epsilon$ for some small $\epsilon > 0$, it is a **near-additive** spanner).*

What if we relax the strict subgraph or stretch requirements?

- **Emulator:** A graph $H = (V, E', \omega)$ approximating distances in G . It is **not restricted to being a subgraph** and may use new weighted edges to "emulate" original paths.
- **Skeleton:** A connected spanning subgraph of G where the **stretch requirement is dropped**, serving merely as an ultra-sparse backbone for connectivity.

Example: A 3-Spanner



- Notice how many edges are removed, yet the shortest path between any two nodes in H is at most 3 times their distance in G .

Checking the Stretch Factor

Proposition 1

H is a t -spanner of G if and only if $d_H(u, v) \leq t \cdot d_G(u, v)$ for all $uv \in E$.

Proof.

The forward direction is obvious. For the reverse direction, let u and v be distinct vertices in V , and let $u_0 u_1 \dots u_m$ be a shortest $u - v$ path in G , where $u_0 = u$ and $u_m = v$. Then note that by assumption,

$$d_H(u, v) \leq \sum_{i=0}^{m-1} d_H(u_i, u_{i+1}) \leq t \cdot \sum_{i=0}^{m-1} d_G(u_i, u_{i+1}) = t \cdot d_G(u, v).$$



Hardness of Optimal Spanners

Given an unweighted graph G , and integers $t, m' \geq 1$, determining if G has a t -spanner containing m' or fewer edges is **NP-complete**, even when t is fixed to be 2.

- **Proof Idea:** Based on a reduction from the Edge Dominating Set (EDS) problem on bipartite graphs.
- **Consequence:** Exact optimization is intractable; thus, our focus shifts entirely to **efficient approximation algorithms**.

Graph Girth and Edge Density

The **girth** of a graph is the length of its shortest cycle.

Maximum Edge Density: Let $\gamma(n, k)$ denote the maximum possible number of edges in an n -vertex graph with girth strictly greater than k .

Proposition 2

An undirected unweighted graph $G = (V, E)$ of girth $> t + 1$ has no proper subgraph that is a t -spanner.

Proof.

- For any edge $uv \in E$, we have $d_G(u, v) = 1$.
- If uv is removed, the shortest alternative path length is exactly one less than the length of the shortest cycle containing uv .
- Since G 's overall girth is $> t + 1$, this new path is strictly $> t$.
- Thus, $d_{G \setminus \{uv\}}(u, v) > t$, and the stretch property is instantly violated.

□

The Moore Bounds

Determining the exact value of $\gamma(n, k)$ remains a major open problem, but asymptotic upper bounds are given by a folklore counting argument:

Proposition 3 (Moore Bounds)

$$\gamma(n, k) = O\left(n^{1 + \frac{1}{\lfloor k/2 \rfloor}}\right)$$

- **Algorithmic Implication:** If an algorithm claims to construct a t -spanner with $|E'|$ edges in the general case, it **cannot** beat the density bound of $\gamma(n, t + 1)$.
- In other words, any bound like $|E'| \leq \gamma(n, t + 1) - 1$ is theoretically impossible.

Erdős' Girth Conjecture

The Erdős' Girth Conjecture states that the Moore Bounds are absolutely tight:

$$\gamma(n, k) = \Omega \left(n^{1 + \frac{1}{\lfloor k/2 \rfloor}} \right)$$

Crucial Roles in Spanner Literature:

- **Ultimate Baseline:** It defines the fundamental theoretical limit on how sparse any spanner could possibly be.
- **Proving Optimality:** Many natural algorithmic questions and optimal size/distortion tradeoffs (like $O(n^{1+1/k})$ edges for $2k - 1$ stretch) are considered "optimal" **only if** this conjecture is assumed to be true.

The Traditional Approach: Greedy Algorithm

The simplest and most basic algorithm for computing a multiplicative α -spanner ($\alpha \geq 1$) is the **greedy algorithm**.

Greedy Spanner

```
 $E' \leftarrow \emptyset$   
 $H \leftarrow (V, E')$   
for each edge  $e = (u, v) \in E$  do  
    if  $d_H(u, v) > \alpha$  then  
         $E' \leftarrow E' \cup \{(u, v)\}$   
        Update  $H$   
return  $H$ 
```

Analysis of the Greedy Algorithm

1. Why is the output a valid α -spanner?

- For any edge $e = (u, v) \in E$, the algorithm discards it **only if** $d_H(u, v) \leq \alpha$.
- Thus, all edges in E inherently satisfy the stretch constraint. By the triangle inequality, this guarantees the required stretch for all vertex pairs.

2. Why does H have no short cycles?

- Suppose H contains a cycle of length $\leq \alpha + 1$.
- Consider the **last edge** $e = (u, v)$ of this cycle that was evaluated.
- Just before e was considered, the rest of the cycle was already in H , forming a path of length $\leq \alpha$ between u and v .
- The algorithm would have **discarded** e ! This contradiction proves $g(H) \geq \alpha + 2$.

Properties of the Greedy Spanner: The Payoff

By design, the greedy algorithm ensures that the resulting α -spanner H contains **no cycles of length $\alpha + 1$ or less**.

- **Girth Consequence:** The girth of H satisfies $g(H) \geq \alpha + 2$.
- **Theoretical Optimality:** Recall Moore's Bound, which states that a graph with girth g has at most $O\left(n^{1+\frac{2}{g-2}}\right)$ edges.
- **The Final Edge Count:** Substituting $g(H) \geq \alpha + 2$ into the bound, the maximum number of edges in our spanner H is strictly bounded by:

$$|E(H)| = O\left(n^{1+\frac{2}{\alpha}}\right)$$

Distributed Models of Computation

In the distributed model, processors reside in vertices of the graph. The processors communicate with their graph neighbors in synchronous rounds. The running time of an algorithm in this model is the number of rounds that it runs.

The LOCAL Model

In the LOCAL model, messages' size is arbitrary.

The CONGEST Model

In each round, messages of bounded length can be sent.

Previous Results in the CONGEST Model

State-of-the-art randomized algorithms in the distributed-CONGEST model:

- **[MPVX15]:** Computes an $O(k)$ -spanner with $O(n^{1+1/k})$ expected edges in $O(k)$ **rounds**.
- **[BS07, Elk07a]:** Computes a $(2k - 1)$ -spanner with $O(k \cdot n^{1+1/k})$ expected edges in k **rounds**.

Theoretical Lower Bound

Under Erdős' girth conjecture, it is proven that **at least k rounds** are strictly required for this task [Elk07a, DGPV08].

Our Contributions

This paper improves the previous state-of-the-art across the entire range of parameters, offering three main results:

- **Unweighted Graphs (CONGEST):** Computes a $(2k - 1)$ -spanner with $O(n^{1+1/k}/\epsilon)$ edges in exactly k rounds. (For $k \geq \log n$, edges tighten to $n(1 + O(\frac{\log n}{\epsilon \cdot k}))$).
- **Ultra-Sparse Skeletons:** The first efficient distributed algorithm yielding an $\tilde{O}(\log n)$ -spanner with $n(1 + o(1))$ edges with probability $1 - o(1)$.
- **Weighted Graphs:** Extends the approach to compute a $(2k - 1)(1 + \epsilon)$ -spanner with $O(n^{1+1/k} \cdot \frac{\log k}{\epsilon})$ edges in expected $O(|E|)$ centralized time.

Probabilistic Setup & Core Parameters

Let $G = (V, E)$ be a graph on n vertices, and let $k \geq 1$ be an integer.

Key Parameters

- $c > 3$: A parameter governing the success probability.
- $\beta = \ln(cn)/k$: The scaling factor.

The Exponential Distribution

Each vertex will sample a random value from $\mathcal{EXP}(\beta)$, which has the density function:

$$f(x) = \begin{cases} \beta \cdot e^{-\beta x} & x \geq 0 \\ 0 & \text{otherwise.} \end{cases}$$

Algorithm: Randomized $(2k - 1)$ -Spanner

```
 $\beta \leftarrow \frac{\ln(cn)}{k}$   
for  $u \in V$  do  
  Draw  $r_u \sim \mathcal{EXP}(\beta)$   
   $u$  broadcasts  $r_u$  to all vertices  $v$  within distance  $k$   
  if  $x$  receives a message from  $u$  then  
     $x$  stores  $m_u(x) = r_u - d_G(u, x)$   
     $x$  stores  $p_u(x)$ , a neighbor on a shortest path from  $u$  to  $x$   
    (break ties arbitrarily if there is more than one)  
  end if  
end for  
 $m(x) \leftarrow \max_{u \in V} m_u(x)$   
for  $x \in V$  do  
   $E' \leftarrow E' \cup C(x) = \{(x, p_u(x)) : m_u(x) \geq m(x) - 1\}$   
end for  
return  $E'$ 
```

Lemma 1: Bounding Candidate Edges

The following lemma is implicit in [MPVX15]. We provide a proof for completeness.

Lemma 4 (1)

Let $d_1 \leq \dots \leq d_n$ be arbitrary values and let $\delta_1, \dots, \delta_n$ be independent random variables sampled from $\mathcal{EX}\mathcal{P}(\beta)$. Define the random variables $M = \max_i \{\delta_i - d_i\}$ and $I = \{i : \delta_i - d_i \geq M - 1\}$. Then for any $1 \leq t \leq n$,

$$\Pr[|I| \geq t] = (1 - e^{-\beta})^{t-1}.$$

Proof of Lemma 1

Proof.

- Denote by $X^{(t)}$ the random variable which is the t -th largest among $\{\delta_i - d_i\}$.
- Then for any value $a \in \mathbb{R}$, if we condition on $X^{(t)} = a$, then the event $|I| \geq t$ is exactly the event that all the remaining $t - 1$ values $X^{(1)}, \dots, X^{(t-1)}$ are at least a and at most $a + 1$.
- Using the memoryless property of the exponential distribution and the independence of the $\{\delta_i\}$, we have that

$$\Pr[|I| \geq t \mid X^{(t)} = a] = (1 - e^{-\beta})^{t-1}.$$

- Since this bound does not depend on the value of a , applying the law of total probability we conclude that

$$\Pr[|I| \geq t] = (1 - e^{-\beta})^{t-1}.$$



Lemma 2: Expected Size of the Spanner

Using Lemma 1, we can bound the expected size of the spanner H .

Lemma 5 (2)

The expected size of H is at most $(cn)^{1/k} \cdot n$.

Proof: Fix any $x \in V$. The event $|C(x)| \geq t$ happens when at least t shifted variables $r_u - d_G(u, x)$ are within 1 of the maximum. By Lemma 1, this probability is at most $(1 - e^{-\beta})^{t-1}$. We conclude that:

$$\mathbb{E}[|C(x)|] = \sum_{t=1}^n \Pr[|C(x)| \geq t] \leq \sum_{t=0}^{\infty} (1 - e^{-\beta})^t = e^{\beta} = (cn)^{1/k}$$

The lemma follows by the linearity of expectation over all n vertices. □

Claims 3 & 4: Bounding the Distances

Now we argue about the stretch. First, we establish high-probability bounds on the sampled values and distances.

Claim 3

With probability at least $1 - 1/c$, it holds that $r_u < k$ for all $u \in V$.

Proof. For any $u \in V$, $\Pr[r_u \geq k] = e^{-\beta k} = 1/(cn)$. By the union bound over n vertices, $\Pr[\exists u, r_u \geq k] \leq 1/c$. $\square$

Assume for now that the event of Claim 3 holds.

Corollary 4

For any $x \in V$, if $u \in V$ is the vertex maximizing $m_u(x)$, then $d_G(u, x) < k$.

Proof. First note that $m(x) \geq m_x(x) \geq 0$, and using Claim 3 we have $r_u < k$. So $0 \leq m(x) = m_u(x) = r_u - d_G(u, x) < k - d_G(u, x)$. $\square$

Claim 5: Shortest Path Containment

Claim 5: For any $u, x \in V$, if x adds an edge to $p_u(x)$, then there is a shortest path P between u and x that is fully contained in H .

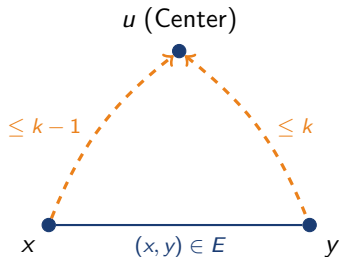
Proof (by induction on $d_G(u, x)$):

- **Base case:** If $d_G(x, u) = 1$, then $p_u(x) = u$, so (x, u) is in H .
- **Step:** Assume true for $t - 1$. Let $d_G(u, x) = t$. We know x added an edge to $y = p_u(x)$ satisfying $d_G(u, y) = t - 1$.
- We claim $m(y) \leq m(x) + 1$. Suppose not; let v maximize $m_v(y)$. By Cor 4, $d_G(v, y) < k$, so $d_G(v, x) \leq k$. Thus x will hear from v , yielding $m_v(x) \geq m_v(y) - 1 = m(y) - 1 > m(x)$, a contradiction.
- By construction, $m_u(x) \geq m(x) - 1$.
- We conclude: $m_u(y) = m_u(x) + 1 \geq m(x) \geq m(y) - 1$.
- So y adds an edge to $p_u(y)$, and by the induction hypothesis we are done. $\square$

Lemma 6: The Stretch Bound

Lemma 6: The spanner H has stretch at most $2k - 1$.

Proof.



- Consider any $(x, y) \in E$. Let u be the vertex maximizing $m(x)$, and w.l.o.g assume $m(x) \geq m(y)$.
- By Cor 4, $d_G(u, x) \leq k - 1$, so $d_G(u, y) \leq k$, and y heard the message of u .
- This implies $m_u(y) \geq m_u(x) - 1 = m(x) - 1 \geq m(y) - 1$.
- So y adds the edge $(y, p_u(y))$ to H .
- By Claim 5, both have shortest paths to u fully contained in H .
- These two paths provide stretch $(k - 1) + k = 2k - 1$. □

Main Theorem

We now state our main theorem, from which we will derive several interesting corollaries in various settings.

Theorem 6 (Theorem 1)

For any unweighted graph $G = (V, E)$ on n vertices, any integer $k \geq 1$, $c > 3$ and $\delta > 0$, there is a randomized algorithm that with probability at least $(1 - 1/c) \cdot \delta / (1 + \delta)$ computes a spanner with stretch $2k - 1$ and number of edges at most

$$(1 + \delta) \cdot \frac{(cn)^{1+1/k}}{c-1} - \delta(n-1).$$

Proof of Main Theorem (Part 1)

Proof.

- Let $\mathcal{Z}$ be the event that $\{\forall u \in V, r_u < k\}$. By Claim 3 we have $\Pr[\mathcal{Z}] \geq 1 - 1/c$.
- Note that conditioning on $\mathcal{Z}$, by Lemma 6 the algorithm produces a spanner $H = (V, E')$ with stretch $2k - 1$. In particular, it must have at least $n - 1$ edges.
- Let X be the random variable $|E'| - (n - 1)$, which conditioned on $\mathcal{Z}$ takes only nonnegative values.
- By Lemma 2 we have $\mathbb{E}[X] = (cn)^{1/k} \cdot n - (n - 1)$.
- We now argue that conditioning on $\mathcal{Z}$ will not affect this expectation by much. Indeed, by the law of total probability, for any t , $\Pr[X = t] \geq \Pr[X = t \mid \mathcal{Z}] \cdot \Pr[\mathcal{Z}]$.

Proof of Main Theorem (Part 2)

Proof (Continued).

- Thus

$$\mathbb{E}[X \mid \mathcal{Z}] \leq \frac{\mathbb{E}[X]}{\Pr[\mathcal{Z}]} \leq \frac{c}{c-1} \cdot \left[(cn)^{1/k} \cdot n - (n-1) \right]. \quad (3)$$

- Recall **Markov's inequality**: for a non-negative random variable Y and $a > 0$,

$$\Pr[Y \geq a] \leq \frac{\mathbb{E}[Y]}{a}$$

- Applying this to X conditional on $\mathcal{Z}$, with $a = (1 + \delta)\mathbb{E}[X \mid \mathcal{Z}]$:

$$\begin{aligned} \Pr[X \geq (1 + \delta)\mathbb{E}[X \mid \mathcal{Z}] \mid \mathcal{Z}] &\leq \frac{\mathbb{E}[X \mid \mathcal{Z}]}{(1 + \delta)\mathbb{E}[X \mid \mathcal{Z}]} \\ &= \frac{1}{1 + \delta}. \end{aligned}$$

Proof of Main Theorem (Part 3)

- We conclude that

$$\begin{aligned}\Pr[(X < (1 + \delta)\mathbb{E}[X \mid \mathcal{Z}]) \wedge \mathcal{Z}] &= \Pr[X < (1 + \delta)\mathbb{E}[X \mid \mathcal{Z}] \mid \mathcal{Z}] \cdot \Pr[\mathcal{Z}] \\ &\geq \left(1 - \frac{1}{1 + \delta}\right) \cdot \left(1 - \frac{1}{c}\right) \\ &= \frac{\delta}{1 + \delta} \cdot \left(1 - \frac{1}{c}\right).\end{aligned}$$

- If this indeed happens, then

$$\begin{aligned}|E'| &= X + n - 1 \\ &\stackrel{(3)}{\leq} (1 + \delta) \cdot \frac{c}{c - 1} \cdot \left[(cn)^{1/k} \cdot n - (n - 1)\right] + n - 1 \\ &= (1 + \delta) \cdot \frac{(cn)^{1+1/k} - (n - 1)}{c - 1} - \delta(n - 1).\end{aligned}$$

□

Implementation Details: Distributed Models

The LOCAL Model

- It is straightforward to implement and takes exactly k rounds to execute.
- In each round, every vertex sends to its neighbors **all** the messages it received so far.

The CONGEST Model (Bandwidth Limited)

- Requires a small variation: in each round, every vertex $v \in V$ will send to all its neighbors **only** the message $(r_u, d_G(u, v))$ for the center u that currently maximizes $m_u(v) = r_u - d_G(u, v)$.
- **Why does this suffice?** Omitting all other messages will not affect the algorithm. If one such omitted message would cause some neighbor of v to add an edge to v , then the message about u will suffice anyway, as the latter has the strictly largest $m_u(v)$ value.
- *(Recall that all vertices start their broadcast simultaneously for k rounds, so dropping omitted messages has no ripple effect on farther vertices.)*

The Goal for Weighted Graphs

Miller et al. [MPVX15] provided a scheme to convert unweighted $O(k)$ -spanners into weighted ones. We adapt this to our tight $(2k - 1)$ -spanner.

The Objective

Convert our unweighted $(2k - 1)$ -spanner algorithm (Theorem 1) into an efficient algorithm for constructing $(2k - 1)(1 + \epsilon)$ -spanners with $O(n^{1+1/k} \cdot \frac{\log k}{\epsilon})$ edges for weighted graphs.

The Scheme of [MPVX15]: Edge Partitioning

- Partition edges into $\lambda = \lceil \log_{1+\epsilon} \omega_{\max} \rceil$ categories:
 $E_t = \{e \in E \mid \omega(e) \in [(1 + \epsilon)^{t-1}, (1 + \epsilon)^t)\}$.
- Define $\ell = \lceil \log_{1+\epsilon} k^c \rceil$ for a sufficiently large constant c .
- Create ℓ distinct graphs $G_j = (V, \hat{E}_j)$, where $\hat{E}_j = \bigcup \{E_t \mid t \equiv j \pmod{\ell}\}$.

The Scheme of [MPVX15]: Well-Separated Graphs

Properties of each Graph G_j :

- The edge weights in G_j are *well-separated*.
- $\hat{E}_j$ is a disjoint union of at most $q = \lceil \lambda/\ell \rceil$ edge sets $E^{(1)}, \dots, E^{(q)}$.
- **Uniformity:** Edge weights within each set $E^{(i)}$ are within a factor of $1 + \epsilon$.
- **Separation:** Weights across sets satisfy $w^{(i)} = w^{(1)}(k^c)^{i-1}$.

The Blackbox Routine

- For each G_j , the scheme constructs a spanner and takes the union of $\ell = O(\frac{\log k}{\epsilon})$ such spanners as the ultimate spanner.

The Contraction Mechanism

How does the scheme construct the spanner for a single G_j ?

- **Step 1:** Run the unweighted spanner routine on $E^{(1)}$ to construct $H^{(1)}$ and a partition $\mathcal{P}^{(1)}$.
- **Step 2:** Contract each cluster of $\mathcal{P}^{(1)}$ (which have unweighted radii at most $k - 1$) into single vertices of $V^{(2)}$.
- **Step 3:** Run the unweighted routine on $(V^{(2)}, E^{(2)})$ to get $H^{(2)}$ and $\mathcal{P}^{(2)}$, contract again, and repeat.
- **Final Spanner:** The output for G_j is $H = \bigcup_{i=1}^q H^{(i)}$.

Stretch Guarantee

The scheme guarantees a stretch of $(1 + \epsilon)(1 + O(k^{-(c-1)}))O(k)$. This holds because weights within categories are uniform up to $1 + \epsilon$, and contracted clusters have weights roughly k^{-c} smaller than the current scale.

The Shortcoming in the Size Analysis

While the stretch analysis of [MPVX15] is sufficient, their size analysis is problematic for our tight bounds.

The "Plugging-in" Problem

- In [MPVX15], every active vertex contributes expected $O(n^{1/k})$ edges per phase, leading to an overall size of $O(n^{1+2/k})$. They fix this by rescaling $k' = 2k$.
- If we plug in our tight $(2k - 1)$ stretch routine, we obtain a $(2k - 1)(1 + \epsilon)^2(1 + k^{-(c-1)})$ -spanner, but with $O(n^{1+2/k})$ edges.
- Rescaling $k' = 2k$ would yield a $(4k - 2)(1 + O(\epsilon))$ -spanner. We would completely lose our tight $(2k - 1)$ stretch!

Solution: We refine their size analysis to show every vertex u contributes expected $O(n^{1/k})$ edges in **all phases altogether**.

Refined Size Analysis: Definitions

Let's track a vertex u across phases $i = 1, 2, \dots, q$:

- $r_u^{(i)}$: The radius u tosses from $\mathcal{EX}\mathcal{P}(\beta)$ in the i -th phase.
- **Candidate Vertex**: A vertex v is a candidate on phase i if its broadcast reaches u no later than time $-r_u^{(i)} + 1$, meaning:

$$-r_u^{(i)} + 1 \geq -r_v^{(i)} + d(v, u)$$

- $X^{(i)}$: The number of such candidate vertices on phase i .
- j : The phase in which u was contracted.
- $\hat{X}^{(i)}$: The total number of candidates u sees between phase i and phase j .

We focus on $\hat{X} = \hat{X}^{(1)}$, the total contribution across all phases. Our goal is to prove: $\Pr(\hat{X} > t) \leq (1 - e^{-\beta})^t$.

Refining the Size Analysis (Probability Bound 1)

By the law of total probability on the first phase:

$$\Pr(\hat{X} > t) = \sum_{t^{(1)=0}^{\infty} \Pr(X^{(1)} = t^{(1)}) \cdot \Pr(\hat{X} > t \mid X^{(1)} = t^{(1)})$$

Case 1: $t^{(1)} > t$

- We have: $\Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}) = (1 - e^{-\beta})^t$.
- *Justification:* The left-hand side is exactly the probability that on the first phase, all the t first candidate vertices v have $-r_v^{(1)} + d(v, u) \geq -r_u^{(1)}$, conditioned on them being candidates.
- Since these are independent shifted exponential random variables, the equation follows strictly from the memoryless property of the exponential distribution.

Refining the Size Analysis (Probability Bound 2)

Case 2: $t^{(1)} \leq t$

$$\begin{aligned}\Pr(\hat{X} > t \mid X^{(1)} = t^{(1)}) &= (1 - e^{-\beta})^{t^{(1)}} \cdot \Pr(\hat{X}^{(2)} > t - t^{(1)} \mid X^{(1)} = t^{(1)}, \hat{X}^{(1)} \geq t^{(1)}) \\ &= (1 - e^{-\beta})^{t^{(1)}} \cdot \Pr(\hat{X}^{(2)} > t - t^{(1)} \mid \hat{X}^{(1)} \geq X^{(1)})\end{aligned}$$

- By conducting backward induction on the phase (base case is last phase q), we obtain:

$$\Pr(\hat{X}^{(2)} > t - t^{(1)} \mid \hat{X}^{(1)} \geq X^{(1)}) \leq (1 - e^{-\beta})^{t - t^{(1)}}$$

- Thus, for any $t^{(1)} \leq t$, it holds that $\Pr(\hat{X}^{(1)} > t \mid X^{(1)} = t^{(1)}) \leq (1 - e^{-\beta})^t$.

Summing over all $t^{(1)}$, we conclude $\Pr(\hat{X} > t) \leq (1 - e^{-\beta})^t$.

Expected Size and Final Theorem for Weighted Graphs

Using the proven probability bound, the expected contribution of every vertex is tightly bounded across all phases:

$$\mathbb{E}(\hat{X}) = \sum_{t=0}^{\infty} \Pr(\hat{X} > t) \leq \sum_{t=0}^{\infty} (1 - e^{-\beta})^t = e^{\beta} = O(n^{1/k})$$

This gives an overall spanner size of $O(n^{1+1/k})$ for each G_j .

Theorem 7 (Theorem 2)

Given a weighted n -vertex graph G , and a pair of parameters $k \geq 1, 0 < \epsilon < 1$, our algorithm computes a $(2k - 1)(1 + \epsilon)$ -spanner of G with $O(n^{1+1/k} \cdot (\log k)/\epsilon)$ edges, in expected $O(|E|)$ centralized time.

Implications of Theorem 1: Centralized Model

Corollary 7 (Standard Centralized Model)

For any unweighted graph G on n vertices and m edges, and any integer $k \geq 2$, there is a randomized algorithm that with high probability computes a spanner with stretch $2k - 1$ and $n^{1+1/k} \cdot \left(1 + \frac{O(\ln k)}{k}\right)$ edges. The running time is $\tilde{O}(k|E|)$.

Proof Idea:

- Apply Theorem 1 with parameters $c = k$ and $\delta = 1/k$.
- The probability of success in a single run is at least $\frac{k-1}{k} \cdot \frac{1}{k+1} \geq \frac{1}{3k}$.
- The number of edges is bounded by:

$$(1 + 1/k) \cdot \frac{(kn)^{1+1/k}}{k-1} \leq n^{1+1/k} \cdot \left(1 + \frac{O(\ln k)}{k}\right).$$

- Repeating the algorithm $O(k \ln n)$ times guarantees the high probability of success. □

Implications of Theorem 1: Distributed Model

Corollary 8 (Distributed Model)

For any unweighted graph G on n vertices, any $k \geq 1$ and $0 < \epsilon < 1$, there is a randomized distributed algorithm that with probability at least $1 - \epsilon$ computes a spanner with stretch $2k - 1$ and $O(n/\epsilon)^{1+1/k}$ edges, within k rounds.

Proof Details:

- Apply Theorem 1 with parameters $c = 3/\epsilon$ and $\delta = 2/\epsilon$.
- The success probability evaluates to at least:

$$(1 - \epsilon/3) \cdot (1 - \epsilon/(\epsilon + 2)) > 1 - \epsilon.$$

- By Theorem 1, plugging in these parameters bounds the number of edges strictly by $O(n/\epsilon)^{1+1/k}$. □

Ultra-Sparse Spanners

We now consider the regime where $k \geq \ln n$, pushing sparsity to the absolute limit.

Corollary 9 (Ultra-Sparse Spanners)

For any unweighted graph G on n vertices, integer $k \geq \ln n$, and parameter $2/k < \epsilon < 1$, there is a randomized algorithm that with probability $\geq 1 - \epsilon$ computes a $(2k - 1)$ -spanner with $n \cdot \left(1 + \frac{O(\ln n)}{\epsilon \cdot k}\right)$ edges.

Proof Sketch:

- Apply Theorem 1 with parameters $c = k$ and $\delta = 2/\epsilon$.
- Success probability is at least $\frac{k-1}{k} \cdot \frac{2/\epsilon}{2/\epsilon+1} \geq 1 - \epsilon$.
- In the regime $k \geq \ln n$, we have $(kn)^{1/k} \leq e^{(2 \ln n)/k} \leq 1 + O(\ln n)/k$.
- Substituting this into Theorem 1 yields the final edge bound. □

Remark 2: The Skeleton Application

The spanner of Corollary 9 can be used as a **skeleton** for the graph.

- **Parameters:** By setting $\epsilon = o(1)$ and $k = \tilde{O}(\log n)$.
- **Result:** We obtain, with probability $1 - o(1)$, a skeleton with exactly $n(1 + o(1))$ edges.
- **Efficiency:** This ultra-sparse backbone is computed in just $\tilde{O}(\log n)$ rounds in the distributed model.
- **Intuition:** It drops the strict stretch requirement in favor of acting as an incredibly sparse connectivity backbone (almost a tree).

Thank You!

References I

[ABS+20] R. Ahmed, G. Bodwin, F. D. Sahneh, K. Hamm, M. J. L. Jebelli, S. Kobourov, and R. Spence.

Graph spanners: A tutorial review.

Computer Science Review, 37:100253, 2020.

[MPVX15] G. L. Miller, R. Peng, A. Vladu, and S. C. Xu.

Improved parallel algorithms for spanners and hopsets.

SPAA '15, pages 192–201, 2015.

[HZ96] S. Halperin and U. Zwick.

Combinatorial algorithms for the Boolean matrix multiplication.

SODA '96, pages 598–607, 1996.

[ES16] M. Elkin and S. Solomon.

Fast constructions of lightweight spanners for general graphs.

SODA '16, pages 883–901, 2016.

[BS07] S. Baswana and S. Sen.

A simple and linear time randomized algorithm for computing sparse spanners in weighted graphs.

Random Struct. Algorithms, 30(4):532–563, 2007.

[Elk07a] M. Elkin.

A near-optimal distributed fully dynamic algorithm for maintaining sparse spanners.

PODC '07, pages 185–194, 2007.

[DGPV08] B. Derbel, C. Gavoille, D. Peleg, and L. Viennot.

On the locality of distributed sparse spanner construction.

PODC '08, pages 273–282, 2008.